

RAPPORT DE PROJET

Interface Graphique - Jeu du Taquin

Auteurs :

Afane • Adjì • Koutchade

Université de Caen Normandie — L2 Informatique

Package : fr.unicaen.l2.afane_adjì_koutchade.taquin

1. Présentation du Projet

Le projet porte sur le développement complet du jeu du Taquin en Java, à la fois en version console et en version graphique. Il s'agit d'un devoir de programmation dans le cadre du cours d'Interface Graphique en L2 Informatique à l'Université de Caen Normandie.

1.1 Qu'est-ce que le jeu du Taquin ?

Le Taquin est un puzzle coulissant classique composé d'une grille de $N \times M$ cases numérotées, avec une case vide. L'objectif est de remettre toutes les tuiles dans l'ordre croissant en déplaçant les tuiles adjacentes à la case vide. Le projet implémente une grille 4×4 (15 tuiles + 1 case vide).

1.2 Fonctionnalités Implémentées

- Deux modes de jeu : Console (terminal) et Graphique (interface Swing)
- Mélange automatique de la grille au démarrage
- Déplacement des tuiles via le clavier (Z/Q/S/D) ou le clic souris
- Compteur de coups affiché en temps réel
- Annulation du dernier coup (bouton Retour)
- Système d'indices (3 indices par partie)
- Mode Image : affichage de photos découpées en morceaux à la place des numéros
- Possibilité de changer d'image parmi 4 images disponibles
- Aperçu de l'image complète dans le panneau latéral
- Détection automatique de la victoire avec message de félicitations



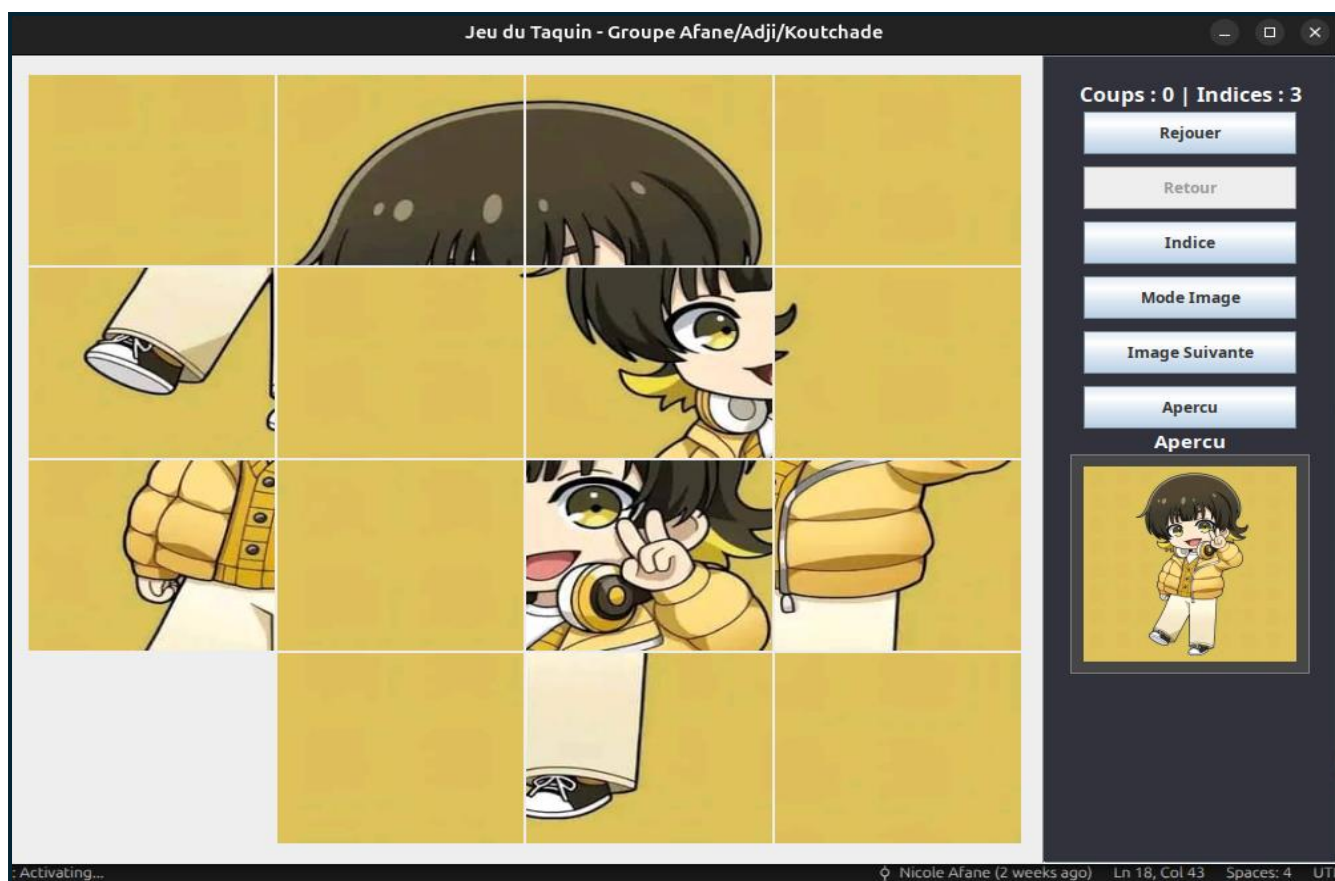


Figure 1 Interface principale du Taquin en mode numérique avec surbrillance des tuiles jouables

1.3 Technologies Utilisees

Technologie	Version / Détail	Utilisation
Java	JDK 21	Langage de programmation principal
Java Swing	javax.swing	Interface graphique (boutons, fenêtre, panels)
Java AWT	java.awt	Gestion des images, couleurs, layouts
javax.imageio	ImageIO	Chargement et découpe des images JPG
Pattern MVC	Conception logicielle	Séparation Modèle / Vue / Contrôleur
Pattern Observer	Conception logicielle	Notification automatique Vue ← Modèle
Git	Contrôle de version	collaboration

Le choix de la bibliothèque **Swing** a été privilégié pour sa robustesse et sa gestion native des composants légers, facilitant la personnalisation des boutons (TuileButton). L'utilisation de **ImageIO** s'est avérée cruciale pour manipuler les flux de données des ressources JPG internes au JAR, permettant ainsi une découpe dynamique de l'image de fond en fonction des dimensions de la grille.

L'implémentation ne se contente pas d'utiliser des bibliothèques Java standards ; elle s'appuie sur des patrons de conception (Design Patterns) industriels pour garantir l'indépendance de la logique de jeu par rapport à l'interface graphique

2. Architecture du Projet (MVC)

Le projet suit strictement le patron de conception MVC (Modèle - Vue - Contrôleur), qui est un standard en développement d'interfaces graphiques. Chaque couche a une responsabilité bien définie et n'a pas connaissance des détails d'implémentation des autres.

Couche	Package	Rôle
Modèle	taquin.modele	Logique du jeu, état de la grille, règles
Vue	taquin.vue	Affichage (console ou Swing), n'agit pas sur le modèle
Contrôleur	taquin.controleur	Reçoit les entrées utilisateur et modifie le modèle
Utilitaires	taquin.util	Interfaces et classes abstraites du pattern Observer, le mécanisme de notification réutilisable pour d'autres projets

Figure 2 Diagramme Taquin

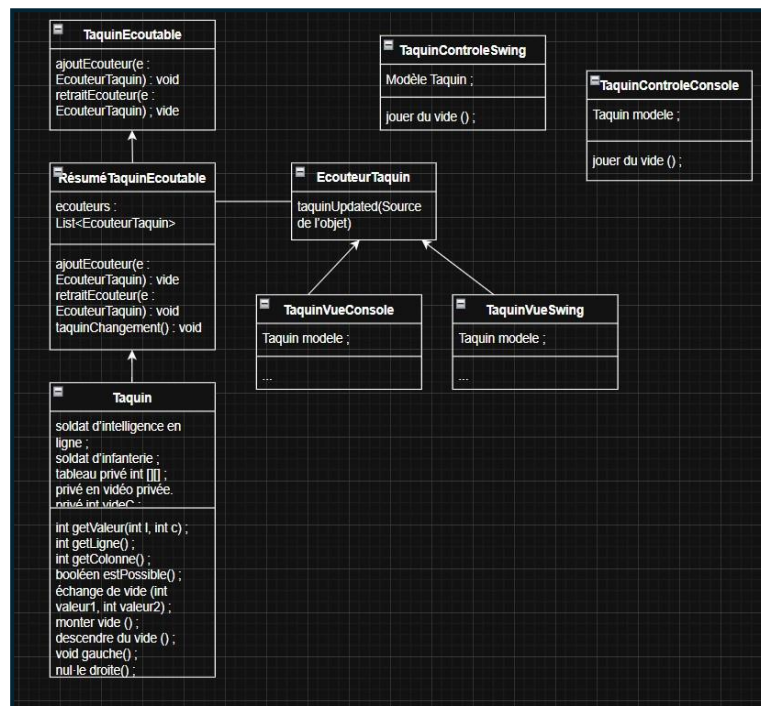
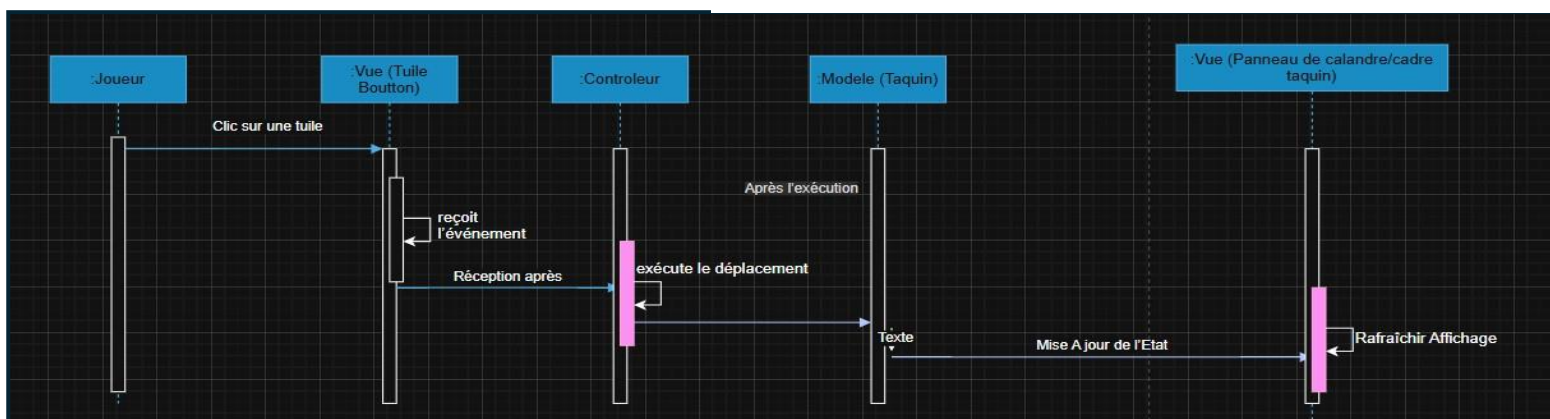


Figure 3 Diagramme de Sequence de la vue



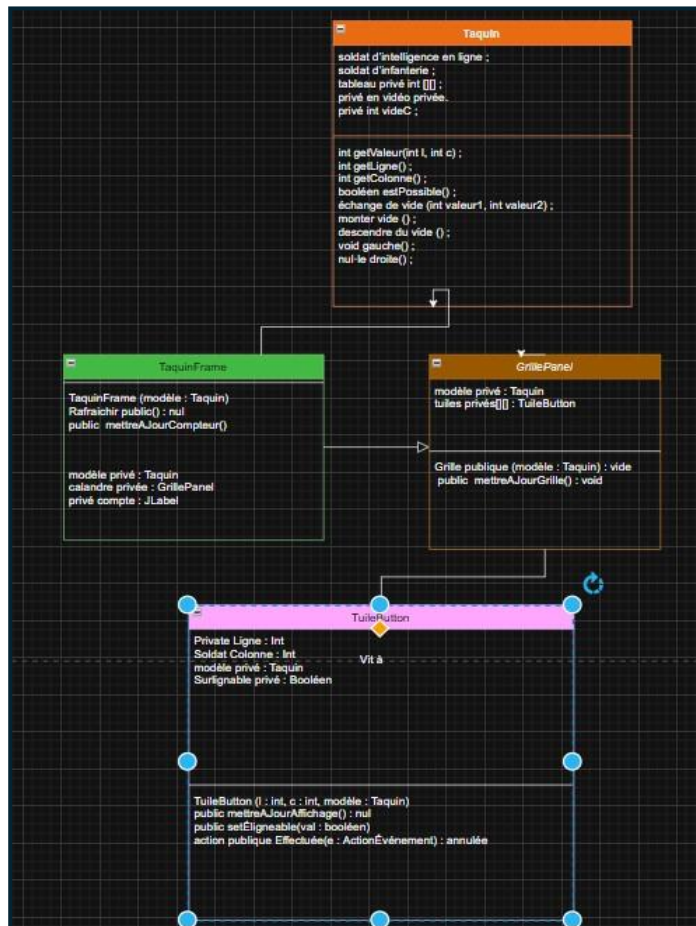


Figure 4 Diagramme de classe de la vue

2.1 Le Pattern Observer

La communication entre le **Modèle** et la **Vue** est l'aspect le plus critique de l'architecture. Nous avons utilisé le pattern **Observer** (via les classes du package `taquin.util`) pour garantir un découplage total.

- **Le Modèle (Sujet/Observable)** : La classe `Taquin` hérite de `AbstractTaquinEcoutable`. Elle possède une liste d'abonnés et ne sait pas s'il s'agit d'une interface Swing ou d'une console. Lorsqu'un mouvement est validé, elle appelle la méthode `taquinChangement()`.
- **La Vue (Observateur)** : Les classes `TaquinFrame` et `TaquinVueConsole` implémentent l'interface `EcouteurTaquin`. Elles s'enregistrent auprès du modèle lors de leur initialisation.
- **La Notification** : Comme l'illustre le **Diagramme de Séquence (Figure 3)**, dès que le modèle change, il "notifie" ses observateurs. La vue reçoit cet événement et déclenche sa propre méthode de rafraîchissement (`rafraichir()` ou `afficher()`).

3. Description Détaillée des Composants

3.1 Le Modèle : `Taquin.java`

C'est le cœur du jeu. Il gère une matrice d'entiers où 0 représente la case vide.

- **Algorithme de mélange** : Pour éviter les configurations insolubles, le mélange ne déplace pas les tuiles au hasard. Il simule 100 mouvements valides successifs à partir d'une grille déjà ordonnée.
- **Gestion de l'historique** : L'annulation du dernier coup est possible grâce à une pile (`Stack<int[][]>`). Chaque mouvement pousse une copie de la grille actuelle sur la pile. Le bouton "Retour" restaure le sommet de cette pile.

3.2 La Vue Graphique : TaquinFrame.java et GrillePanel.java

- **Gestion des images** : La classe GrillePanel utilise ImageIO pour charger des ressources JPG. L'image est découpée en 16 sous-images. Un calcul de coordonnées permet d'associer à chaque valeur de tuile le bon fragment d'image.
- **Interactivité** : Les tuiles adjacentes à la case vide sont détectées dynamiquement. En mode numérique, elles arborent une couleur distincte pour guider l'utilisateur.

4. Difficultés Rencontrées et Choix Techniques

Le développement de ce projet a présenté plusieurs défis techniques, tant sur le plan de la programmation Swing que sur la gestion des ressources système.

4.1 Gestion des Ressources et du Classpath

L'une des difficultés majeures a été de s'assurer que les images du jeu soient correctement chargées, que le programme soit lancé depuis un IDE ou via un fichier **JAR** autonome.

- **Problème** : L'utilisation de chemins de fichiers relatifs classiques ("src/images/...") échoue une fois le projet compressé en JAR.
- **Solution** : Nous avons utilisé le ClassLoader de Java (getResourceAsStream), ce qui permet d'extraire les images directement depuis l'archive. Cela garantit que le jeu est "portable" et ne nécessite aucun dossier externe pour fonctionner.

4.2 Ergonomie et Contrôles Clavier (ZQSD vs W)

L'implémentation des contrôles au clavier a nécessité une réflexion sur l'ergonomie.

- **Problème** : Selon la disposition du clavier (AZERTY ou QWERTY), la touche de direction "Haut" change. De plus, il y a souvent une confusion entre "déplacer la case vide vers le haut" et "déplacer la tuile située en bas vers le haut".
- **Solution** : Nous avons implémenté une gestion d'événements KeyEvent supportant à la fois **Z** (pour AZERTY) et **W** (pour QWERTY/International). Nous avons également veillé à ce que l'action soit intuitive : presser "Haut" déplace physiquement une tuile vers le haut pour combler le vide.

4.3 Optimisation de l'Espace Disque (Quota Étudiant)

Lors de la phase finale de livraison, nous avons été confrontés à une saturation du quota d'espace disque sur les serveurs de l'université.

- **Problème** : L'accumulation des fichiers de compilation (.class) et de l'historique Git empêchait la génération du JAR final.
- **Solution** : Nous avons migré temporairement le répertoire de travail vers un espace disque moins restreint et mis en place une procédure de nettoyage systématique des fichiers binaires (rm -rf bin/) avant chaque livraison.

4.4 Threading et Fluidité (EDT)

Pour éviter que l'interface graphique ne "gèle" lors du chargement ou du mélange de la grille, nous avons respecté la règle du **Single Thread Model** de Swing.

- **Solution** : Toutes les instantiations de fenêtres sont encapsulées dans SwingUtilities.invokeLater(). Cela garantit que toutes les mises à jour visuelles se produisent sur l'**Event Dispatch Thread**, évitant ainsi des bugs d'affichage aléatoires.

5. Conclusion

Ce projet nous a permis de mettre en pratique les concepts fondamentaux de la programmation orientée objet et des interfaces graphiques. L'utilisation du pattern **MVC** s'est avérée indispensable pour gérer la complexité croissante du jeu (ajout du mode image, historique des coups).

Le résultat final est une application robuste, respectant les standards de conception Java, et offrant une expérience utilisateur complète (aide au jeu, retour arrière, personnalisation visuelle). Ce travail constitue une base solide pour comprendre le développement d'applications événementielles plus complexes.